

Computer Networks II

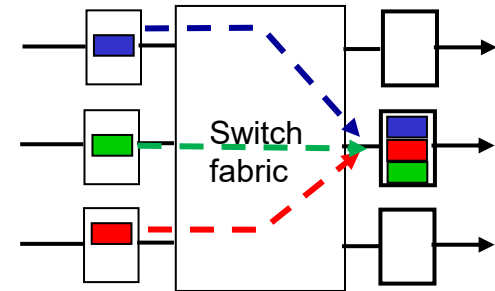
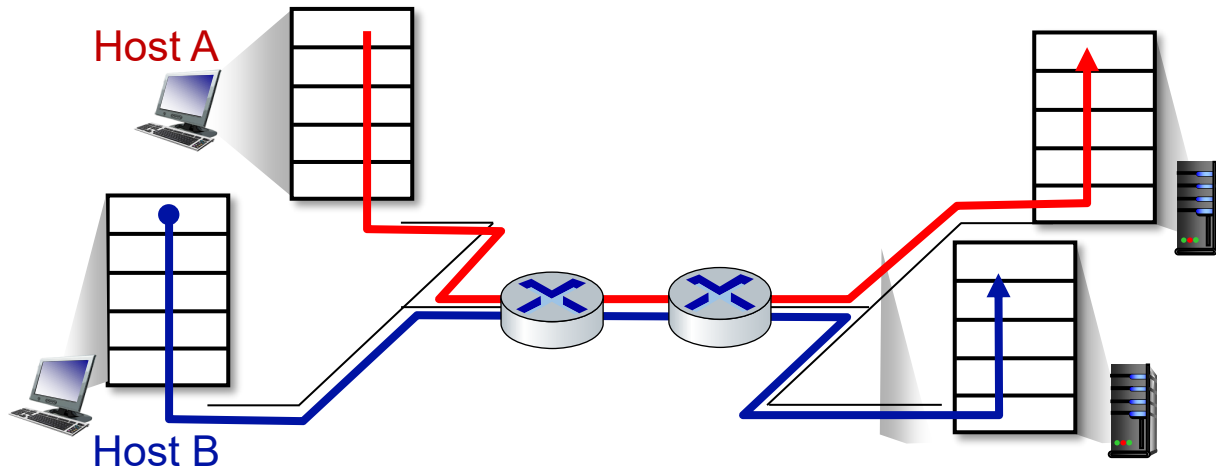
TCP Congestion Control

TCP Tahoe and TCP Reno

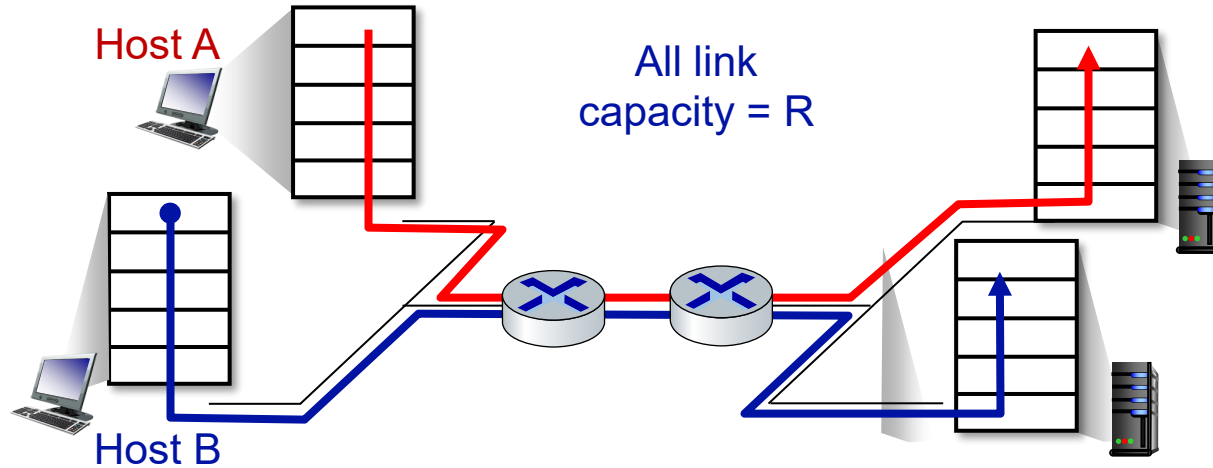
Amitangshu Pal
Computer Science and Engineering
IIT Kanpur

Network Congestion

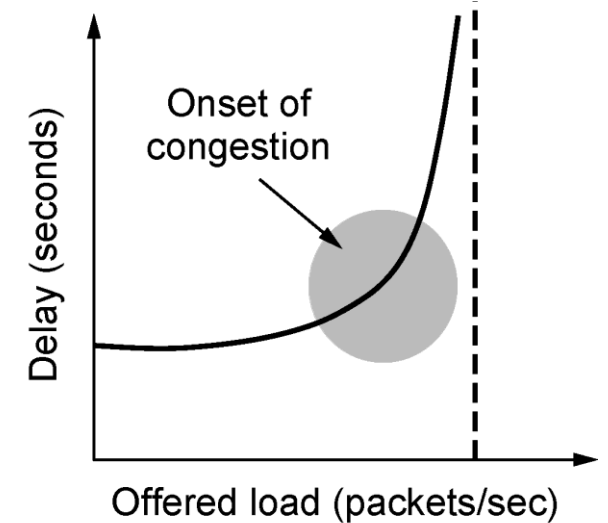
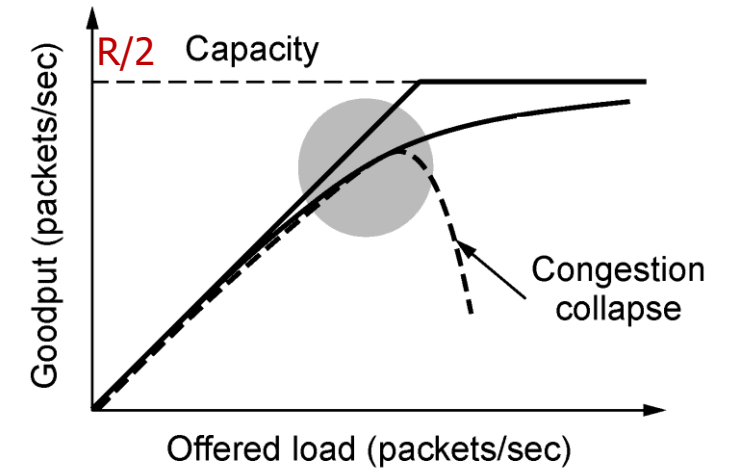
- Routers have input/output queue
 - These queues get filled up → leads to congestion



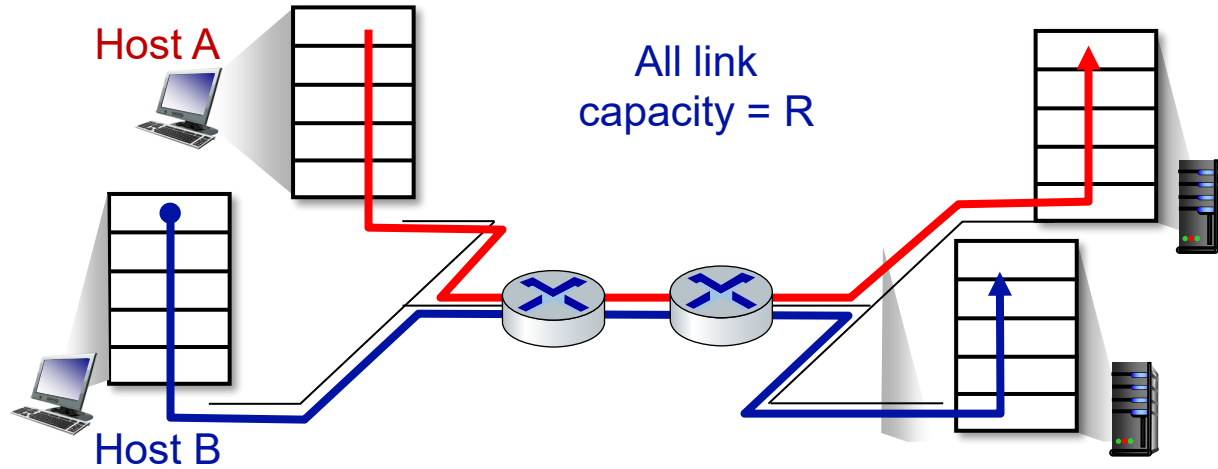
Effects of Congestion



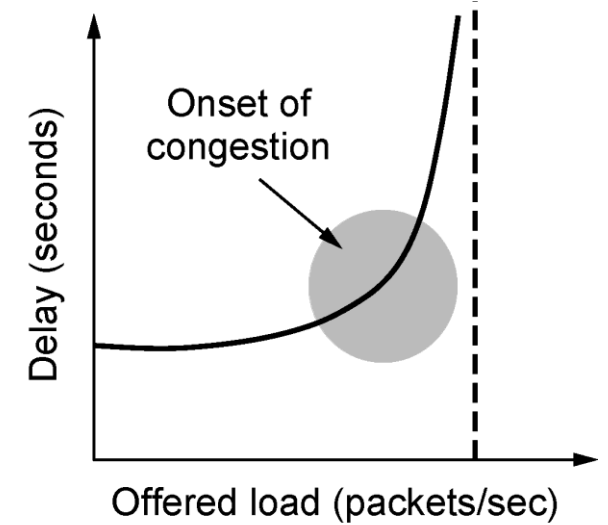
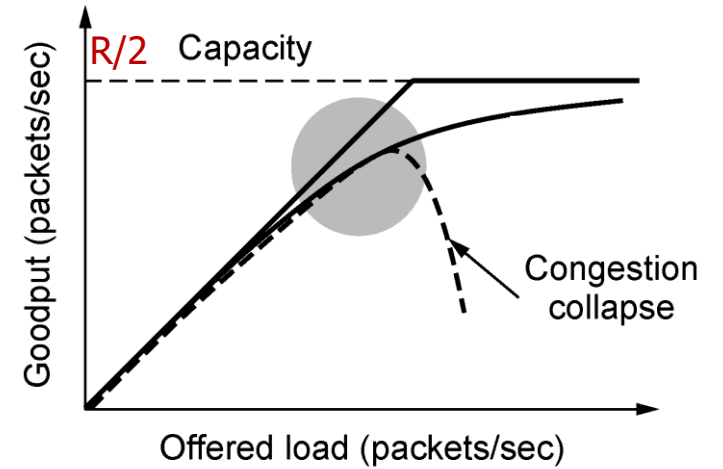
- **Retransmitting undelivered packets:** Packets can be lost, dropped at router due to full buffers
- **Retransmitting timeout packets:** Sender timers can time out prematurely, sending two copies, both of which are delivered
 - Decrease maximum achievable **goodput**



Effects of Congestion

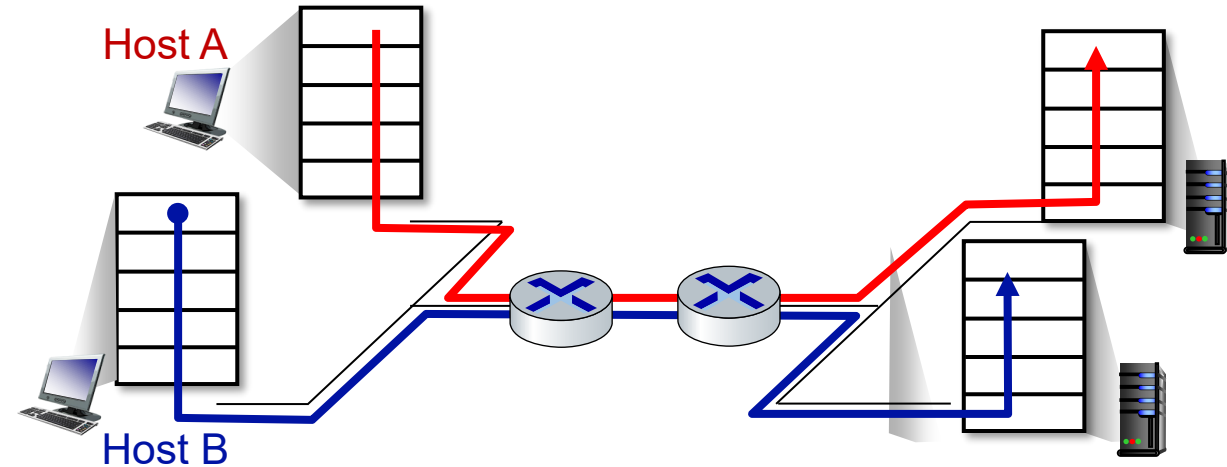


- We want to operate just before the **onset of congestion**
 - Avoid congestion
 - Most capacity is used
 - Needs to be fair across competitive flows



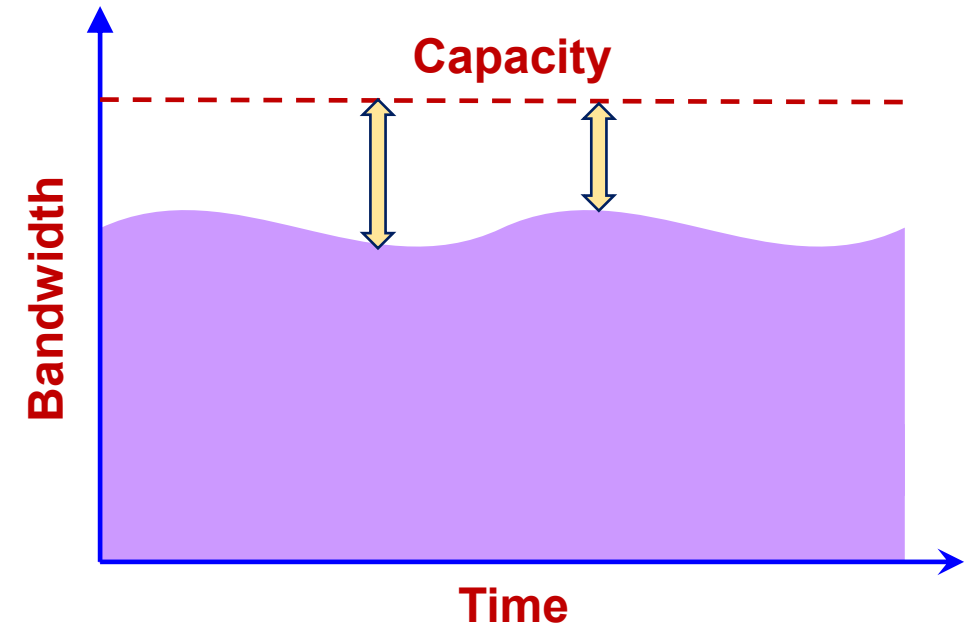
Challenges of Congestion Control

- How the senders will know whether there is a congestion or not?
- No explicit feedback from network
 - Congestion **inferred** from observed loss, delay



Challenges of Congestion Control

- The senders need to adapt/tune the sending rates to match the available bandwidth of the bottleneck link
 - Sending rate= function (available bandwidth of the bottleneck link)
- Which link has the lowest available bandwidth?
 - Sender does not have the global picture
- Available bandwidth varies with time
 - Number of flows and their offered load is dynamic
 - Bottleneck link can also vary

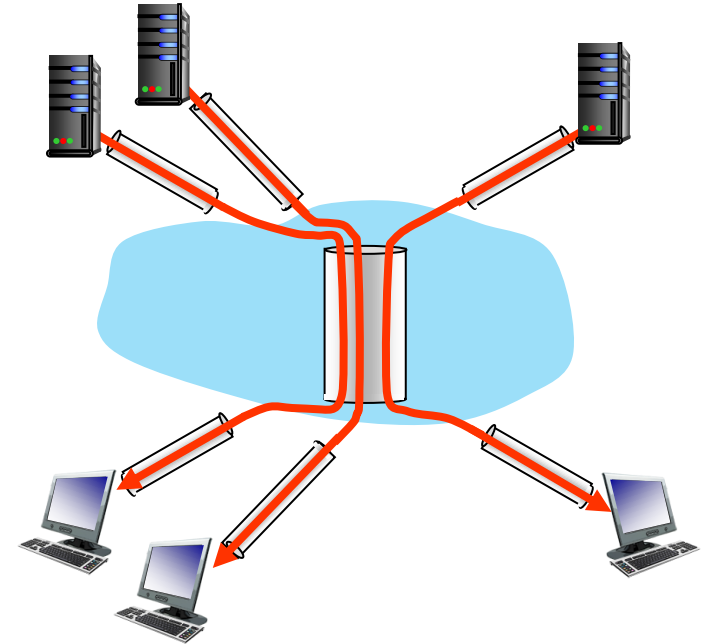


Challenges of Congestion Control

- The senders need to adapt/tune the sending rates to match the available bandwidth of the bottleneck link

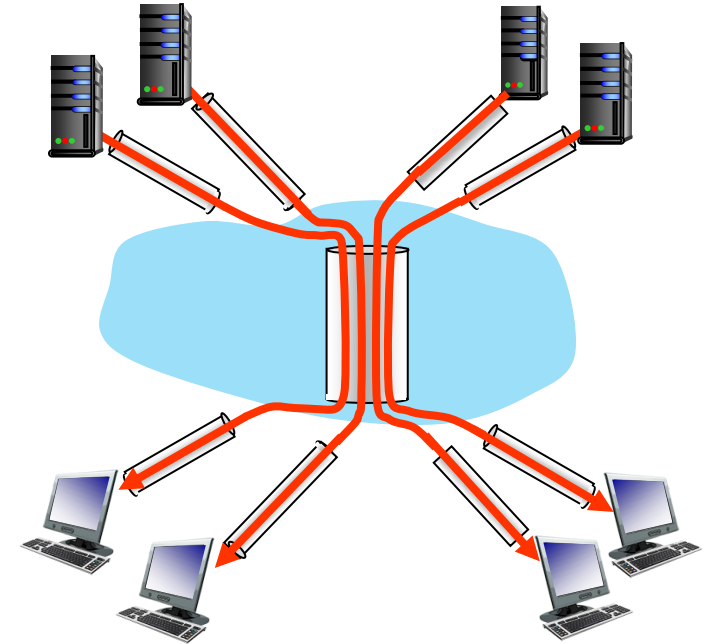
- Sending rate = function (available bandwidth of the bottleneck link)

- Which link has the lowest available bandwidth?
 - Sender does not have the global picture
- Available bandwidth varies with time
 - Number of flows and their offered load is dynamic
 - Bottleneck link can also vary



Challenges of Congestion Control

- The senders need to adapt/tune the sending rates to match the available bandwidth of the bottleneck link
 - Sending rate= function (available bandwidth of the bottleneck link)
- Which link has the lowest available bandwidth?
 - Sender does not have the global picture
- Available bandwidth varies with time
 - Number of flows and their offered load is dynamic
 - Bottleneck link can also vary



TCP Congestion Control

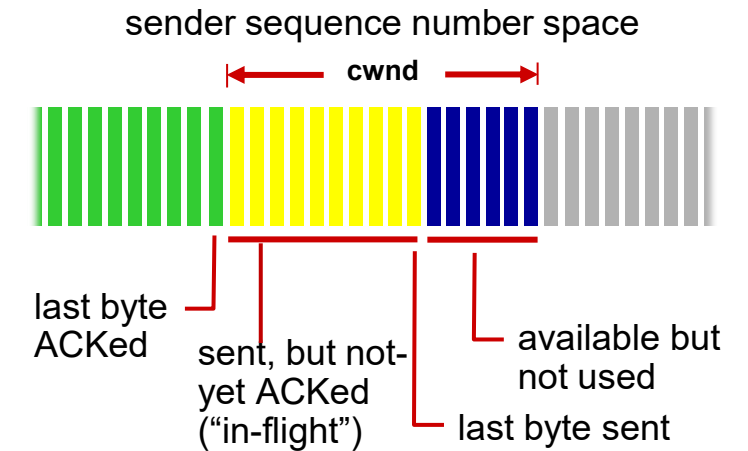
- Senders need to adapt/tune the sending rate
 - How many bytes are inside the network → called **congestion window** or **cwnd**

■ TCP sender limits transmission: $\text{LastByteSent} - \text{LastByteAcked} \leq \text{cwnd}$

- TCP sending behavior : send cwnd bytes, wait RTT for ACKS, then send more bytes

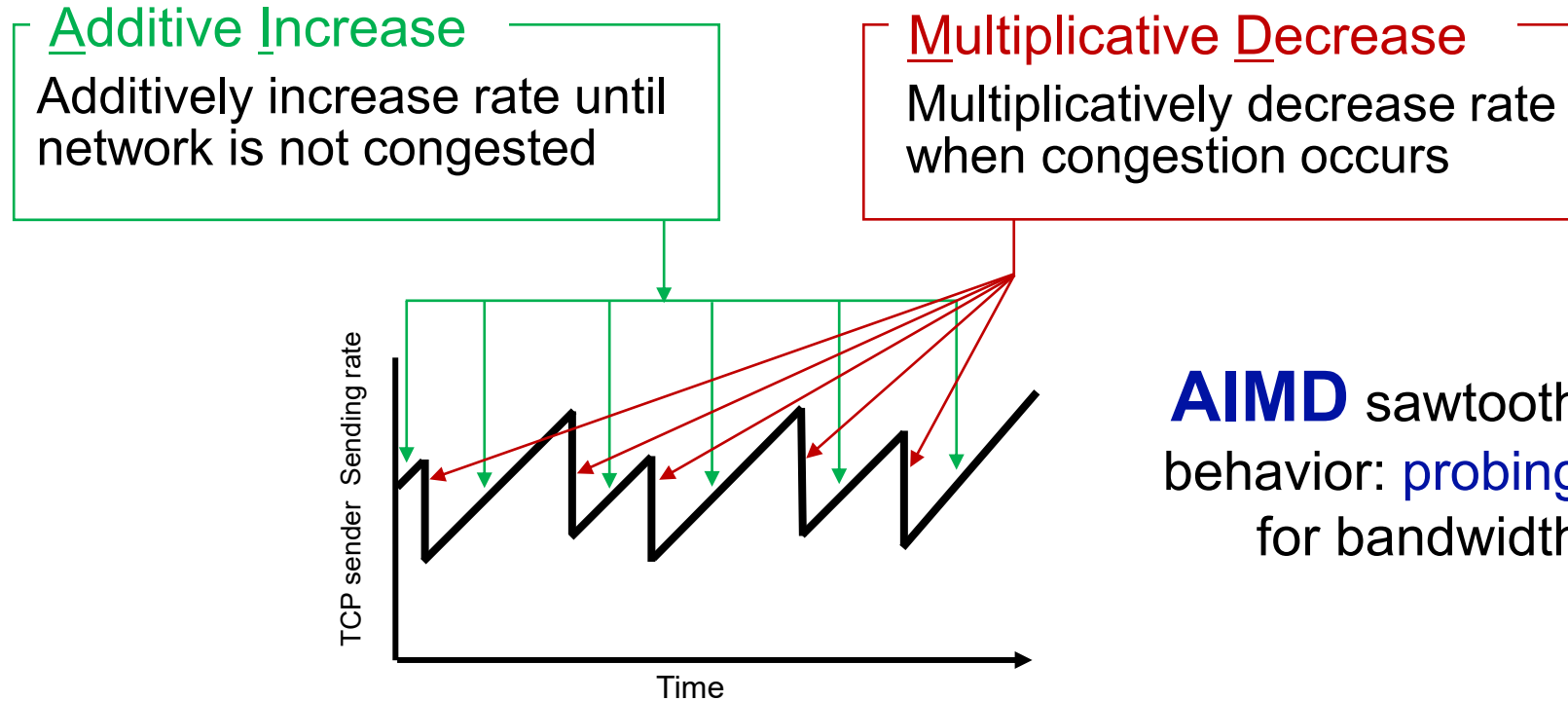
$$\text{TCP rate} \approx \frac{\text{cwnd}}{\text{RTT}} \text{ bytes/sec}$$

- cwnd is dynamically adjusted in response to observed network congestion
→ Implementing **TCP congestion control**



TCP Congestion Control: AIMD

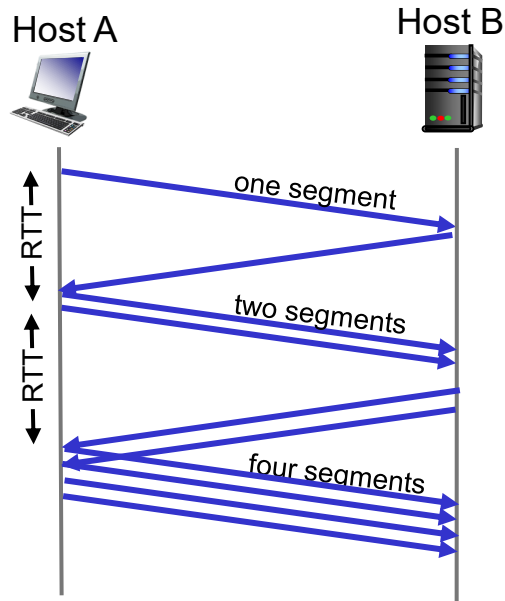
- **Approach:** Senders can increase sending rate until packet loss (congestion) occurs, then decrease sending rate on loss event



TCP Tahoe

TCP slow start

- Assumption: 1 MSS = 1 Byte



Slow start (cwnd = 1 MSS)

CW = 1

1 2 3 4 5 6 7 8

CW = CW + 1 = 2

1 2 3 4 5 6 7 8

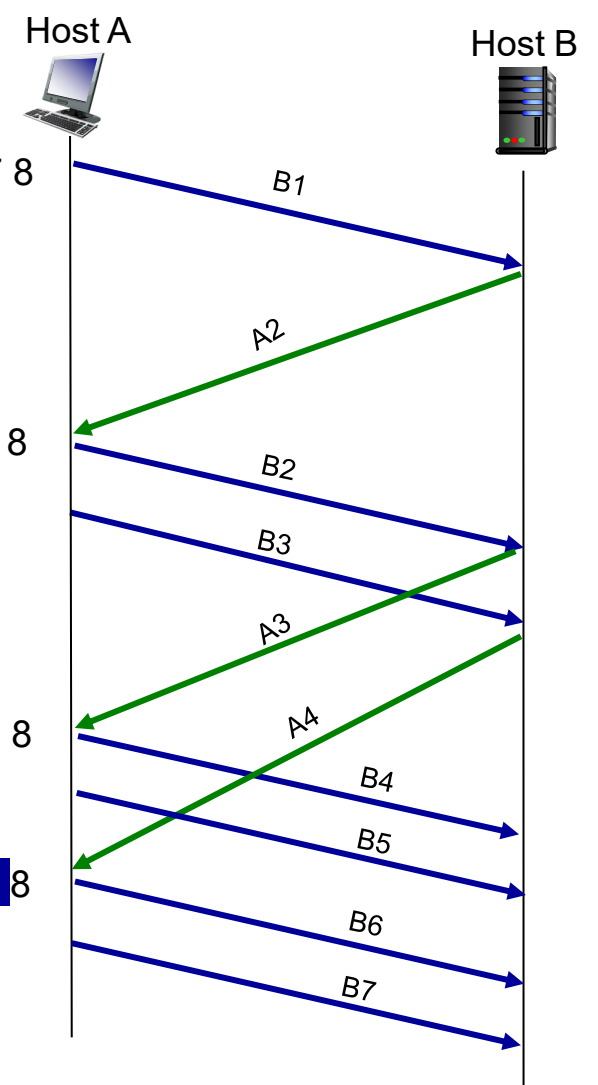
CW = CW + 1 = 3

1 2 3 4 5 6 7 8

CW = CW + 1 = 4

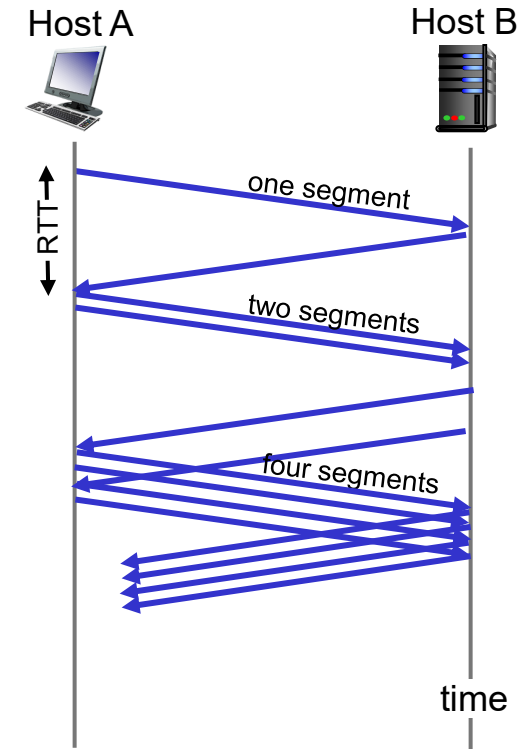
1 2 3 4 5 6 7 8

CW is doubling in every RTT → exponential increase of CW



TCP Slow Start

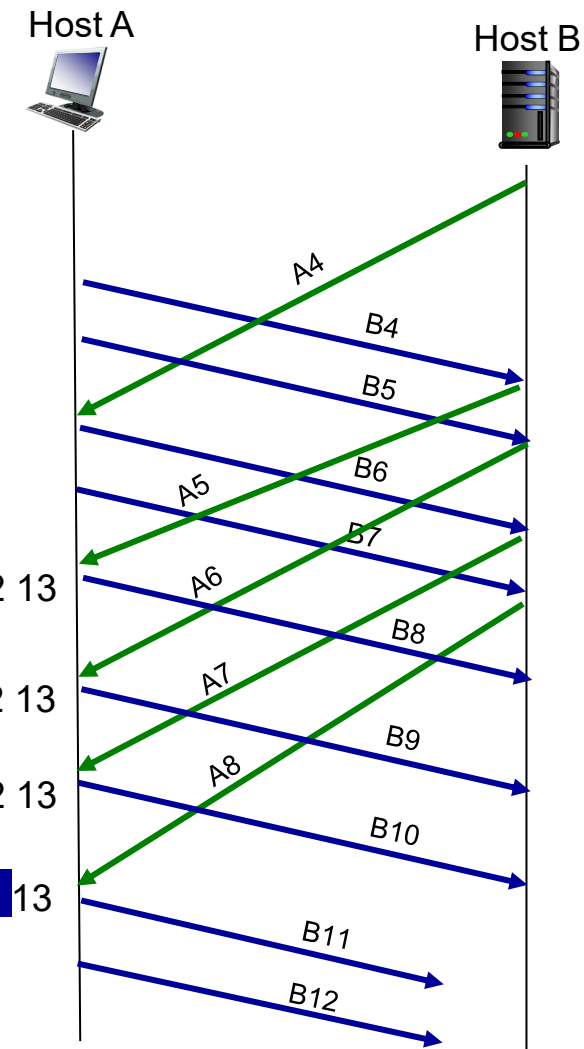
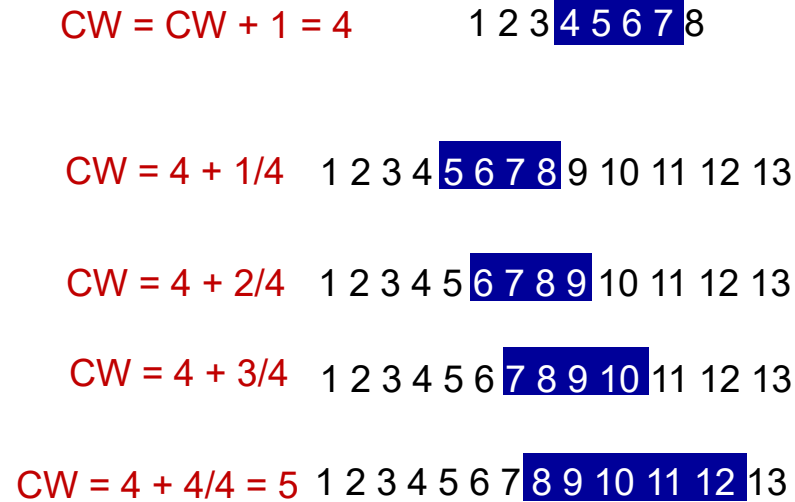
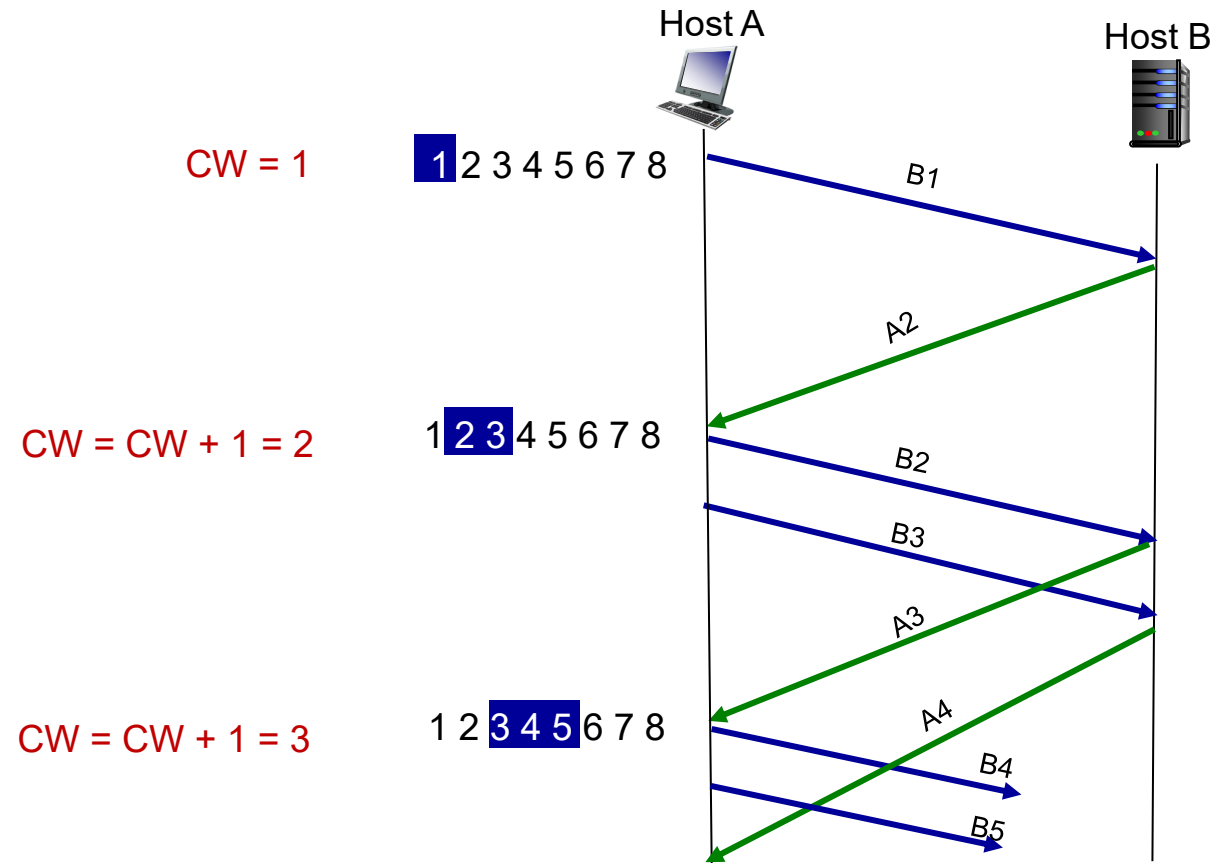
- When connection begins, increase rate exponentially until first loss event:
 - Initially **cwnd** = 1 MSS
 - Double **cwnd** every RTT
 - Done by incrementing **cwnd** for every ACK received
- **Summary:** Initial rate is slow, but ramps up exponentially fast



TCP Congestion Avoidance

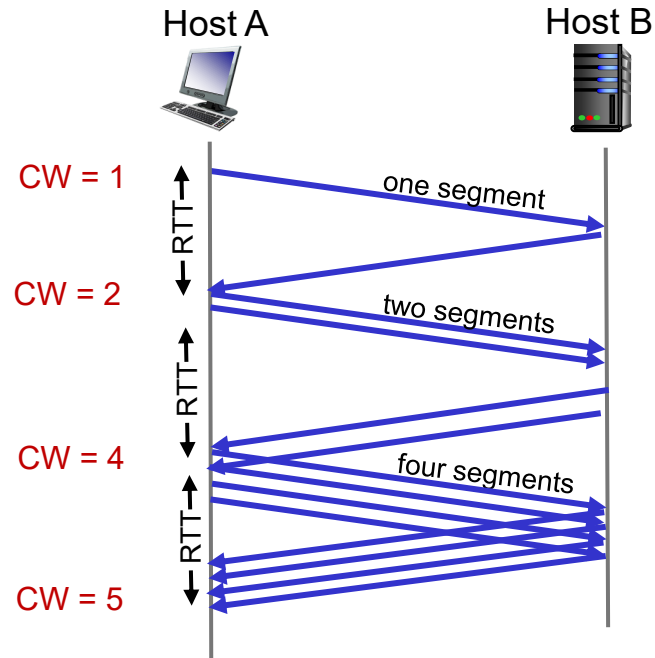
- After ssthresh, reduce the rate of CW increase

$$\begin{aligned} \text{Upon every ACK} \rightarrow \\ \text{CW} &= \text{CW} + \text{MSS} \times \frac{\text{MSS}}{\text{cwnd}} \\ &= \text{CW} + \frac{1}{\text{CW}} \end{aligned}$$



TCP Congestion Avoidance

- After ssthresh, reduce the rate of CW increase



$$\begin{aligned} \text{Upon every ACK} \rightarrow \\ CW &= CW + MSS \times \frac{MSS}{cwnd} \\ &= CW + \frac{1}{CW} \end{aligned}$$

$$CW = CW + 1 = 4 \quad 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8$$

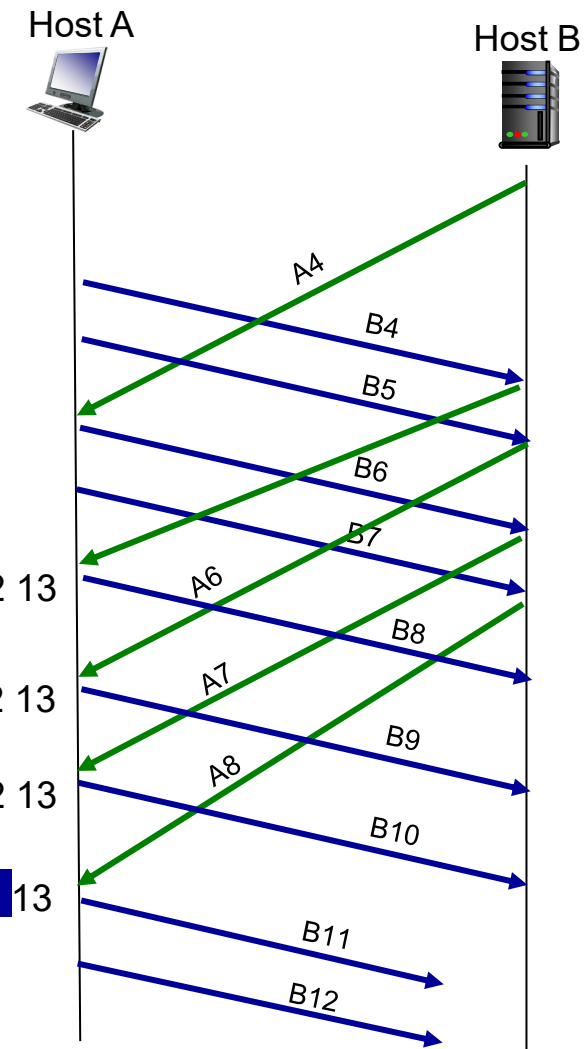
$$CW = 4 + 1/4 \quad 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9 \ 10 \ 11 \ 12 \ 13$$

$$CW = 4 + 2/4 \quad 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9 \ 10 \ 11 \ 12 \ 13$$

$$CW = 4 + 3/4 \quad 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9 \ 10 \ 11 \ 12 \ 13$$

$$CW = 4 + 4/4 = 5 \quad 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9 \ 10 \ 11 \ 12 \ 13$$

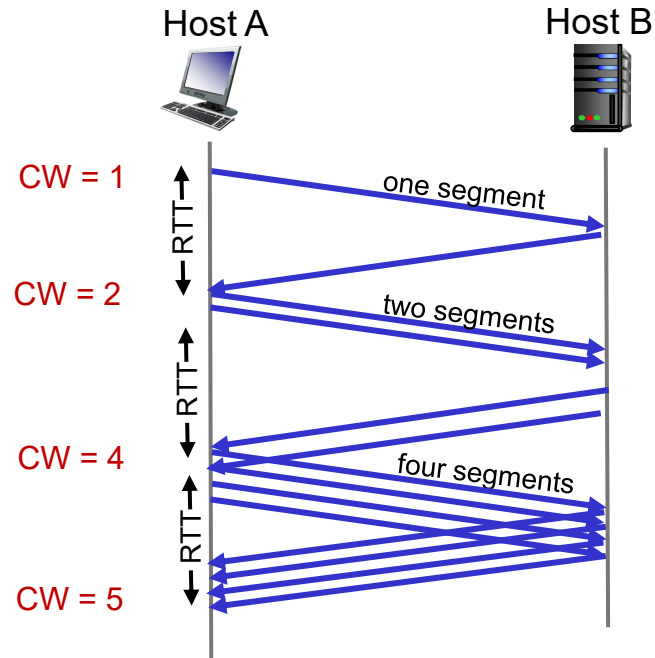
After 1 RTT, the CW is approx. increasing by 1 MSS → Congestion avoidance



TCP Congestion Avoidance

- After ssthresh, reduce the rate of CW increase

$$\begin{aligned} \text{Upon every ACK} \rightarrow \\ \text{CW} &= \text{CW} + \text{MSS} \times \frac{\text{MSS}}{\text{cwnd}} \\ &= \text{CW} + \frac{1}{\text{CW}} \end{aligned}$$

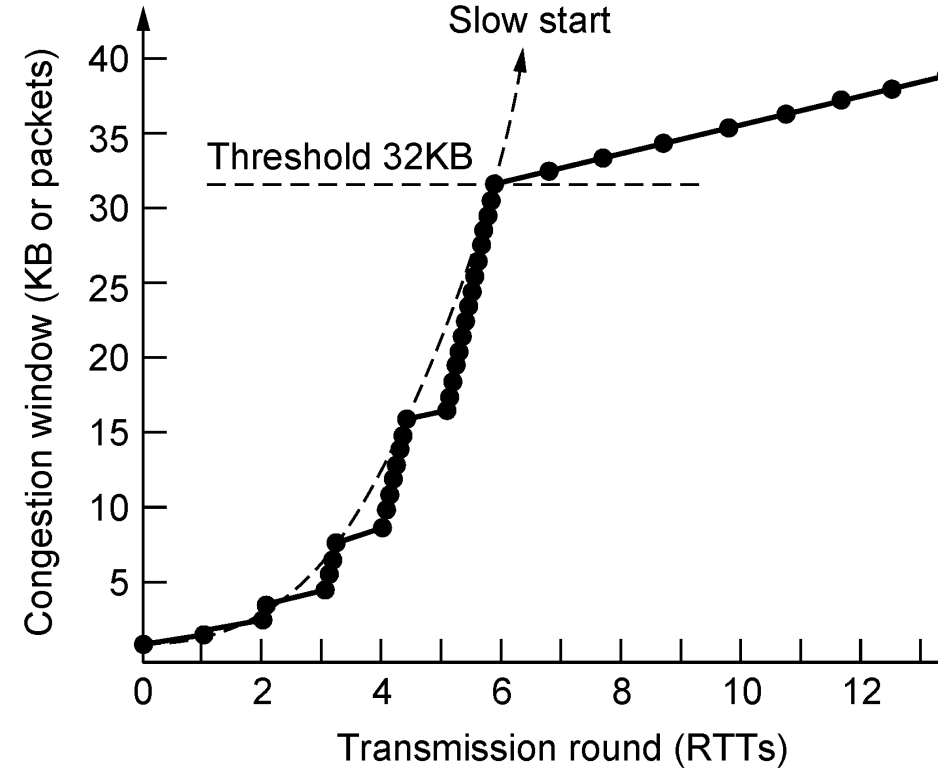
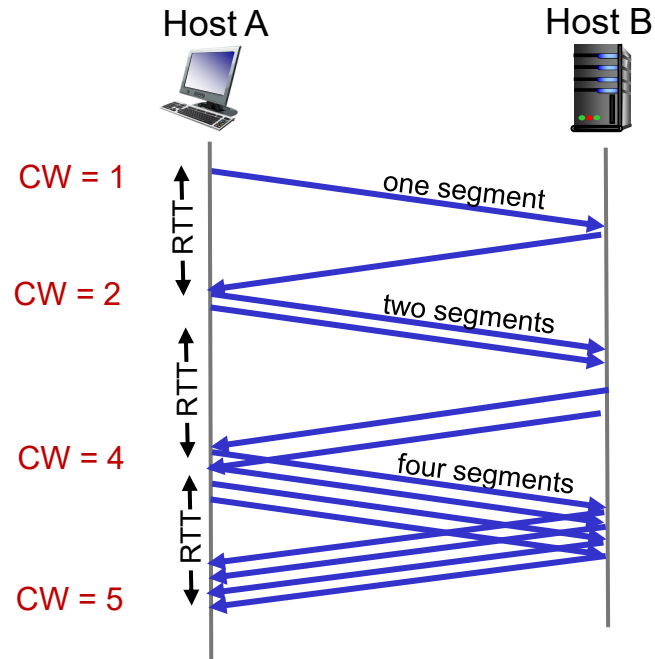


If MSS = 1460 Bytes, CW = 14600 Bytes
→ $\text{CW} = \text{CW} + \text{MSS} \times \frac{1}{10}$
∴ CW will be increased by 1 MSS after all 10 segments will be received ≈ after 1 RTT

After 1 RTT, the CW is approx. increasing by 1 MSS → Congestion avoidance

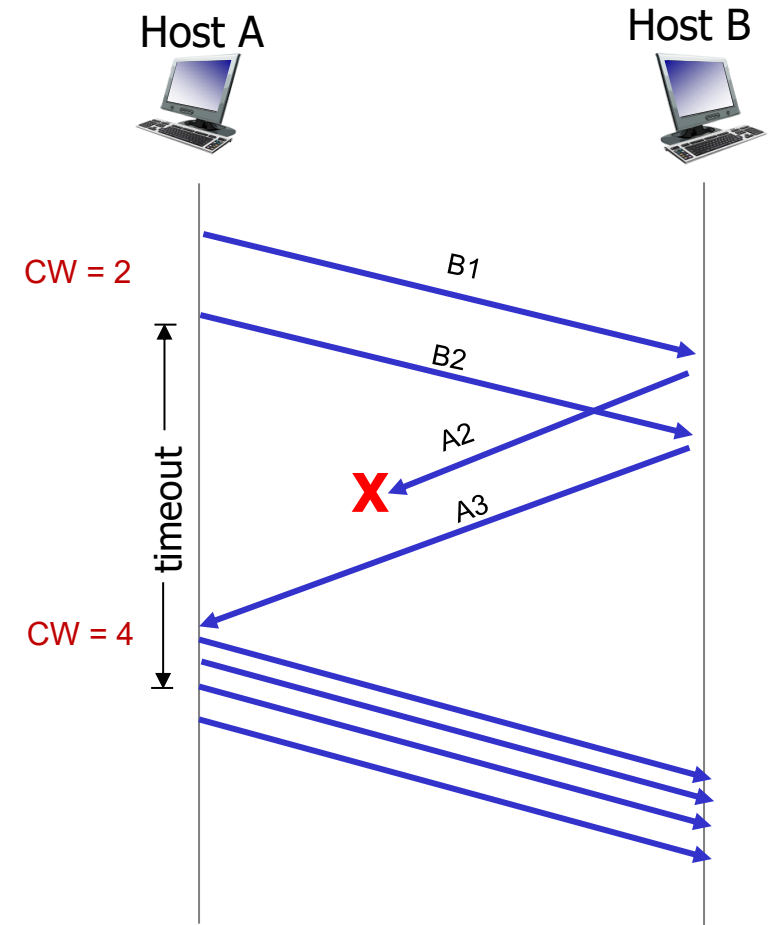
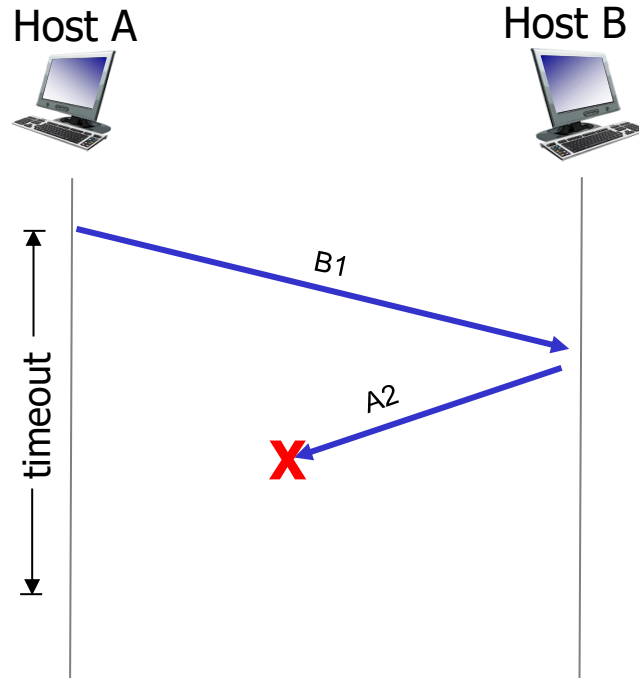
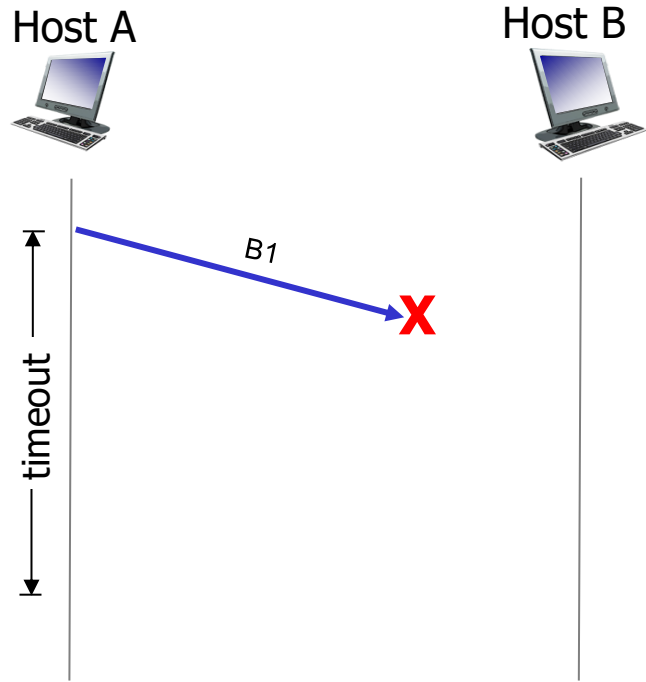
TCP Congestion Avoidance

- After ssthresh, reduce the rate of CW increase



After 1 RTT, the CW is approx. increasing by 1 MSS → Congestion avoidance

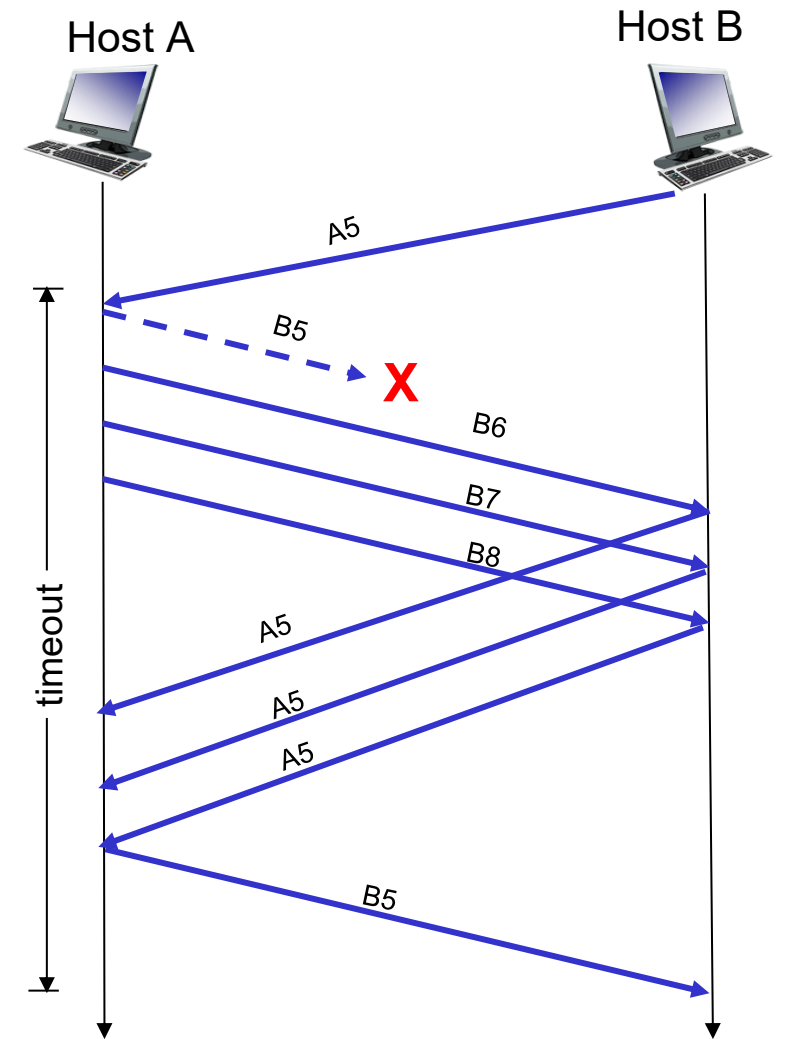
Packet/ACK Drop



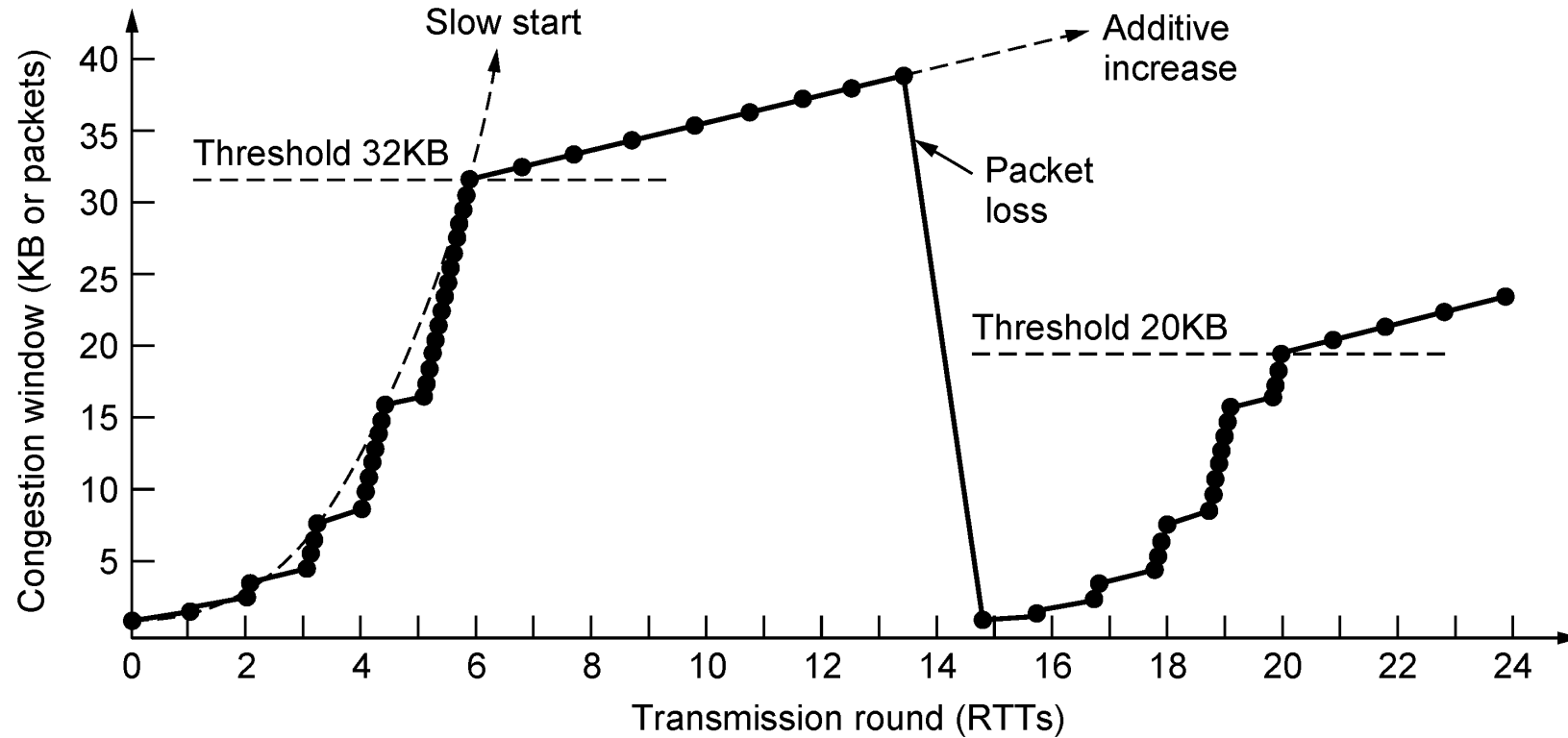
TX infers a packet loss $\rightarrow$ goes to slow start $\rightarrow$ $\begin{cases} ssthresh = \frac{CW}{2} \\ CW = 1 \end{cases}$

TCP Fast Retransmit

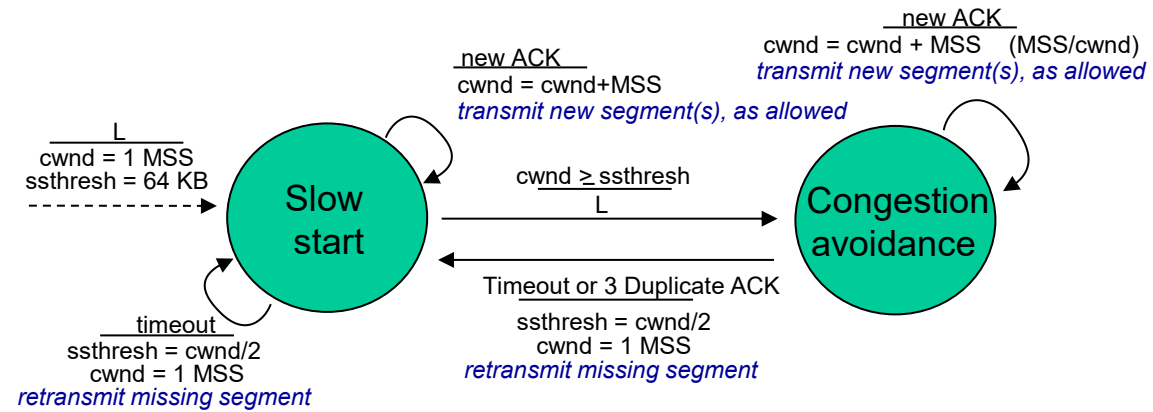
- If sender receives **3 duplicate ACKs**:
 - Likely that unACKed segment lost, so don't wait for timeout
 - Resend unACKed segment with smallest seq #



TCP Tahoe: Slow start followed by additive increase

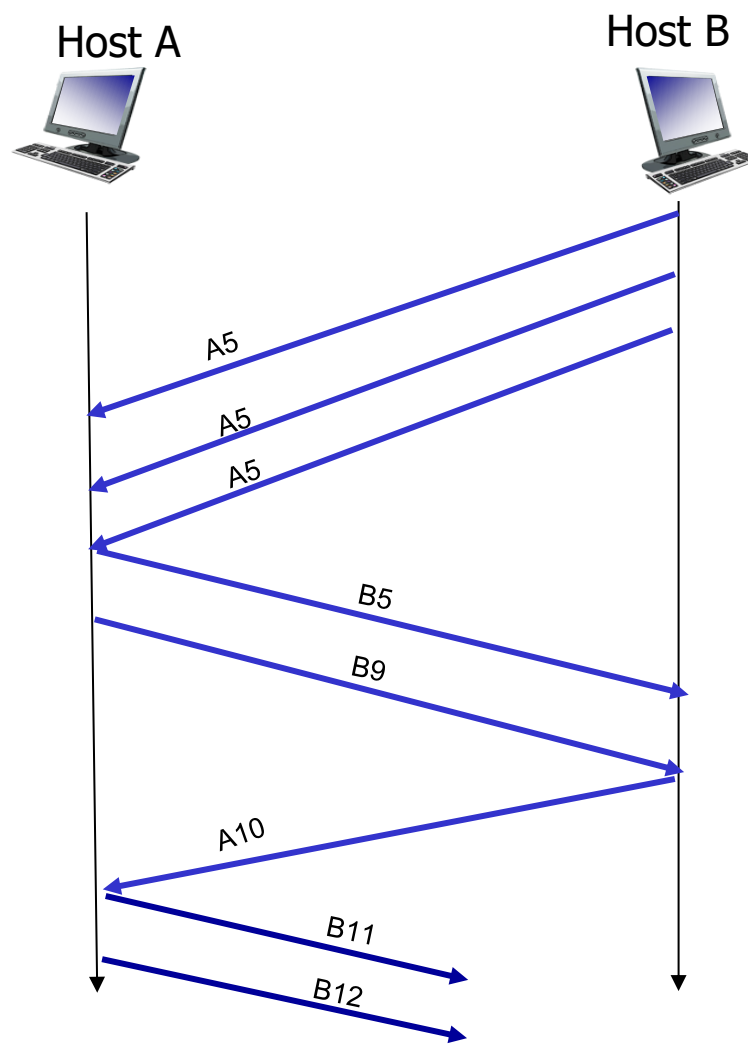
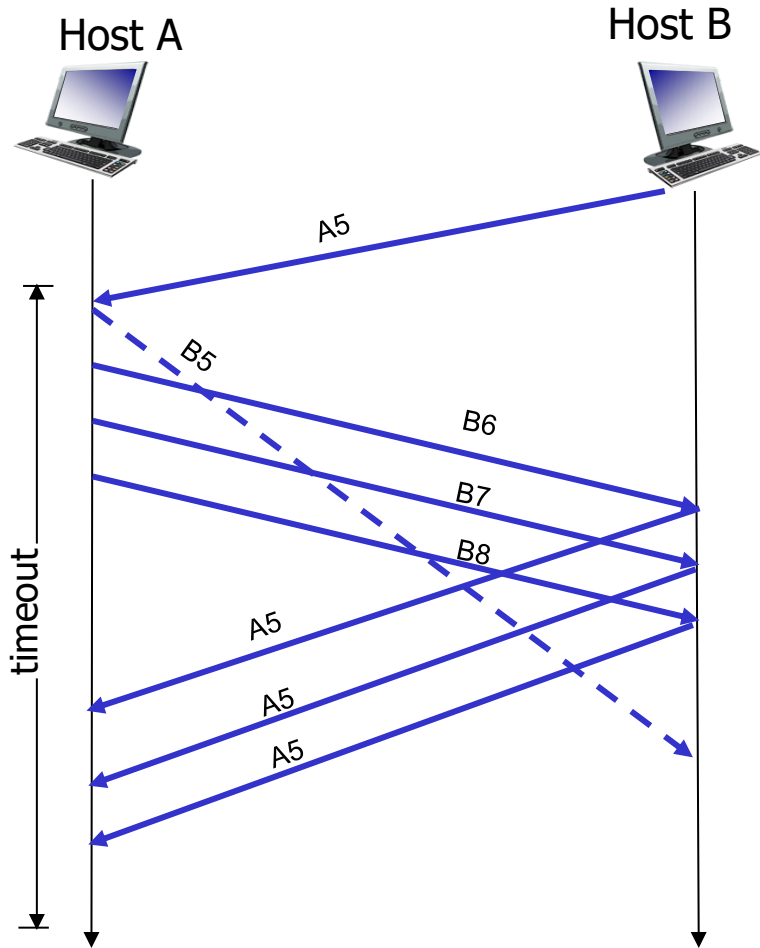


TCP Tahoe



TCP Reno

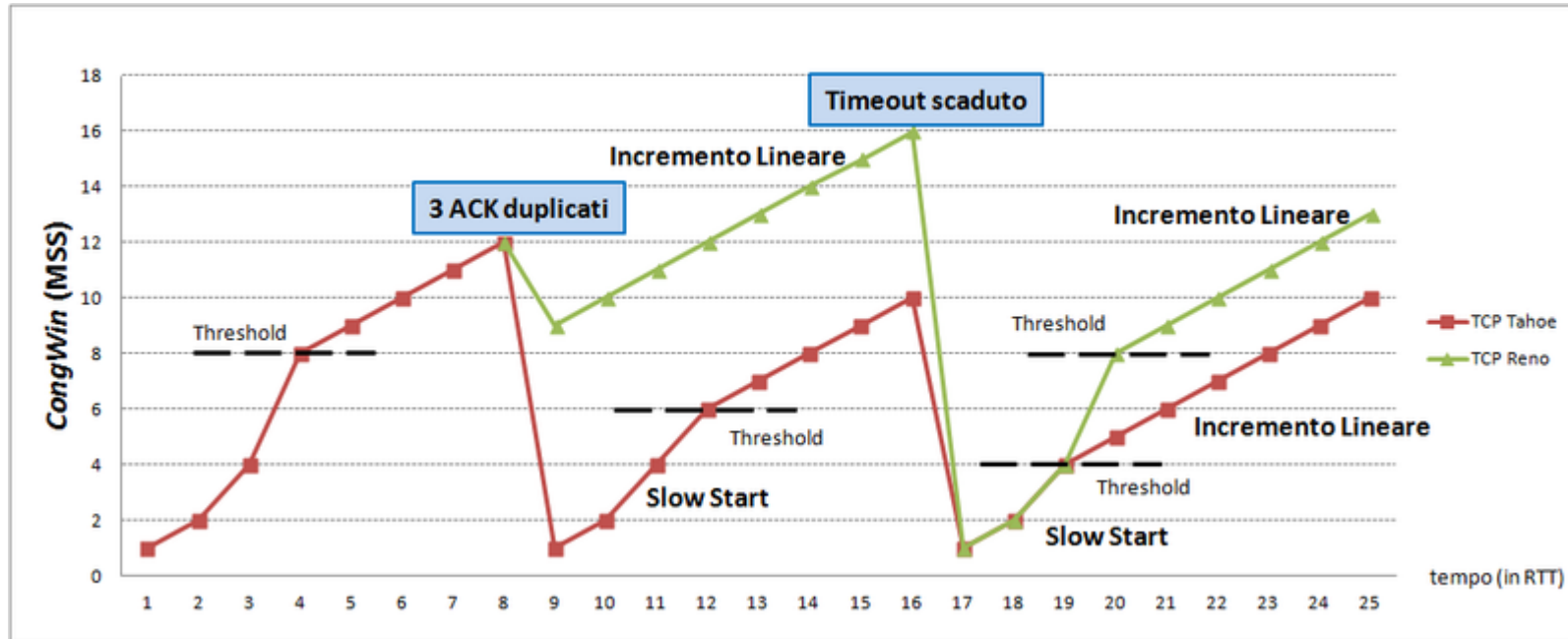
Fast Recovery



3 DupACKs
→ Trigger Fast Retransmit

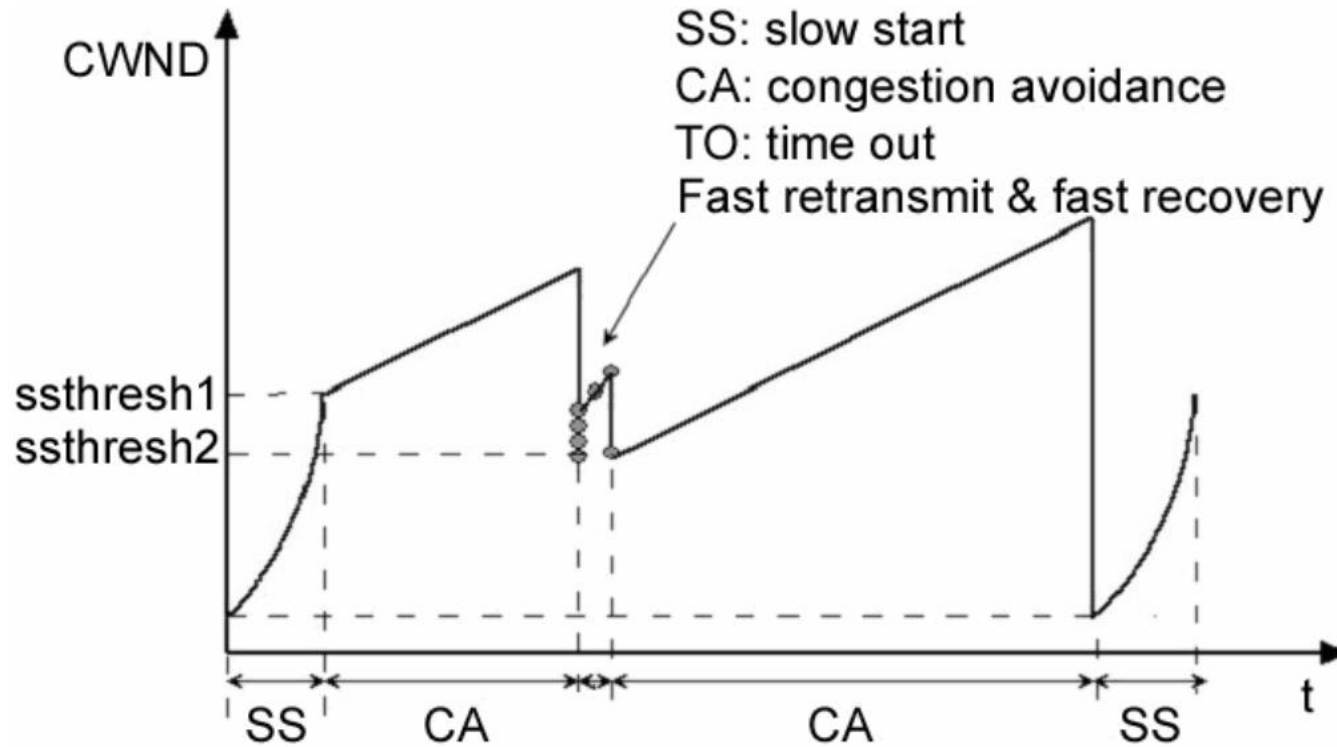
1. $ssthresh = CW/2$
2. Transmit missing segment that is definitely causing DupACKs
3. $CW = ssthresh + 3$
4. Transmit new segments of CW allows
5. More DupACKs:
 $CW = CW + 1$
Transmit new segments if CW allows
6. Normal ACK:
 $CW = ssthresh$
Start congestion avoidance

TCP Reno vs TCP Tahoe



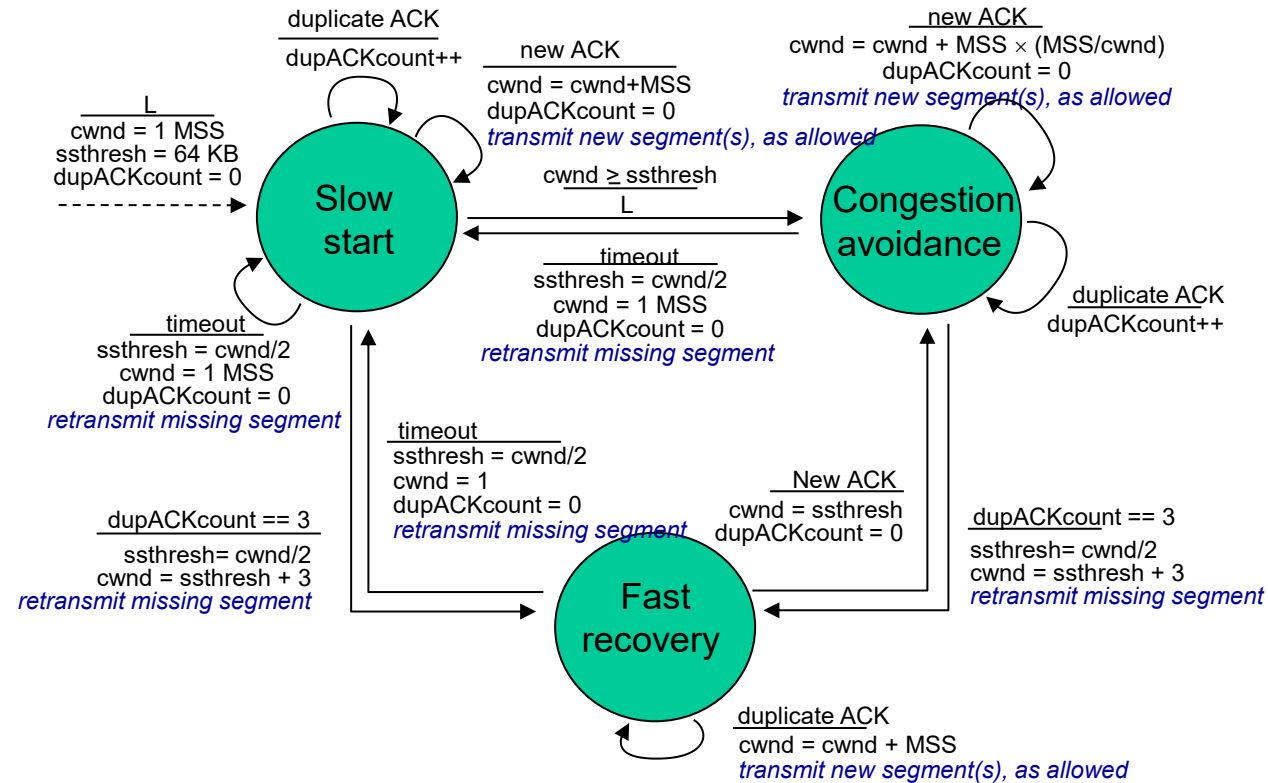
Src: https://commons.wikimedia.org/wiki/File:CongWin_in_TCP_Tahoe_e_Reno.png

TCP Reno vs TCP Tahoe



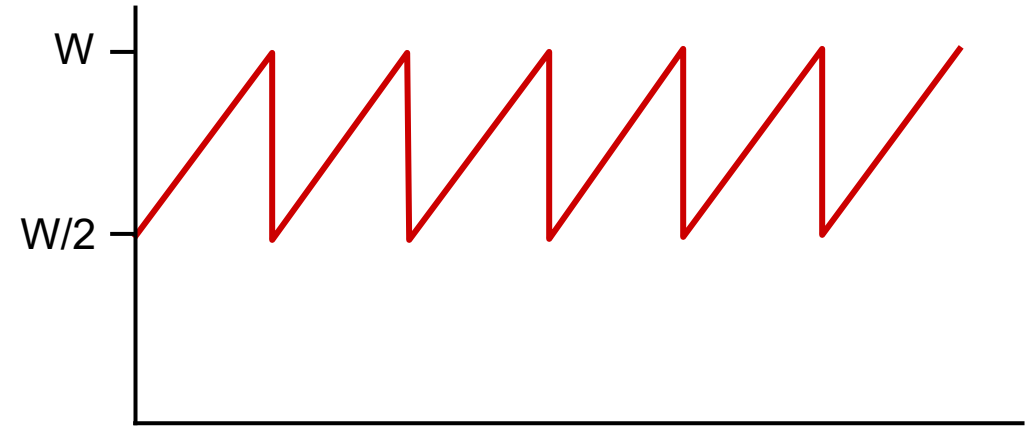
Src: https://www.sfu.ca/~ljilja/papers/WM-195.PacketControl_edited.pdf

TCP Reno: TCP Tahoe + Fast recovery



TCP Throughput

- Avg. TCP throughput as function of window size, RTT?
 - Ignore slow start, assume always data to send
- W: window size (measured in bytes) where loss occurs
 - Avg. window size (# in-flight bytes) is $\frac{3}{4} W$
 - Avg. throughput is $\frac{3}{4}W$ per RTT



$$\text{Avg TCP throughput} = \frac{3}{4} \frac{W}{\text{RTT}} \text{ bytes/sec}$$

Summary

□ Network congestion control:

- Challenges of congestion
 - TCP congestion control
 - TCP Tahoe and TCP Reno
-